

Universidad San Pablo De Guatemala
Facultad de Ciencias Empresariales
Escuela de Ingeniería
Ingeniería en Sistemas y Ciencias de la Computación



Lenguaje Chapin (.gt) — Versión 1.0

Curso Compiladores
Ing. Luis Alberto Pérez Estrada
Sexto Semestre

Grupo #5
Ismael Alejandro Liquez Muñoz # 2400064
Luis Angel Franco Cruz #2400497

Guatemala, 12 de septiembre de 2026

Contrato del Lenguaje Chapin (.gt) — Versión 1.0

1. Descripción del lenguaje

Campo	Valor
Nombre	Chapin
Extensión de archivo	.gt
Propósito	Lenguaje de programación imperativo, pequeño y educativo, cuyas palabras reservadas se basan en chapinismos guatemaltecos. Expresa operaciones aritméticas, lógicas, entrada/salida y control de flujo con una sintaxis cercana al habla popular de Guatemala.
Paradigma	Imperativo, estructurado
Tipos base	pisto (entero), quetzal (real)

2. Tokens

2.1 Palabras reservadas

Palabra	Chapinismo original	Significado en Chapin
va	"va pues" (de acuerdo)	Declaración de variable
pisto	dinero en general	Tipo entero
quetzal	moneda nacional (tiene centavos)	Tipo real
aguas	"¡aguas!" (alerta/cuidado)	Emitir un mensaje de solicitud (prompt)
clavar	"clavarse" (meterse de lleno)	Leer un valor de entrada
shute	persona metiche/curiosa	Imprimir/mostrar una expresión
brega	"bregar" (luchar, darle vueltas)	Ciclo (equivalente a while)
cabal	"está cabal" (exacto/correcto)	Condicional (equivalente a if)
sino	alternativa	Alternativa del condicional (equivalente a else)

2.2 Identificadores

Nombres de variables definidos por el usuario. No pueden empezar con dígito ni ser palabra reservada.

2.3 Números

- Enteros (pisto): secuencia de dígitos (0-9).
- Reales (quetzal): dígitos, punto decimal, dígitos (3.14, 0.5).

2.4 Cadenas

Texto entre comillas dobles, usado en aguas y shute.

2.5 Operadores

Categoría	Símbolos
Asignación	=
Aritméticos	+ - * /
Relacionales	< > <= >= == !=

2.6 Delimitadores

Símbolo	Uso
()	Agrupación de expresiones, condiciones y argumentos de clavar/shute
{ }	Bloques de código
;	Fin de instrucción
,	Reservado para futuras extensiones (ej. funciones)

2.7 Comentarios

- Una sola línea: // comentario
- Multilínea: /* comentario en varias líneas */

3. Patrones (expresiones regulares para Flex)

```
DIGITO      [0-9]
LETRA       [a-zA-Z_]
ID           {LETRA}{LETRA}{DIGITO}*
ENTERO      {DIGITO}+
REAL        {DIGITO}+."{DIGITO}+
CADENA      \"([^\n])*\"
ESPACIO      [ \t\r]+
SALTO_LINEA      \n
COMENTARIO_1L  \"//\".*
COMENTARIO_ML  \"/*\"([^\n]|\"/\"|\"/*\")*\"/\"

/* Palabras reservadas (se reconocen antes que ID) */
"va"        VA
"pisto"      PISTO
"quetzal"    QUETZAL
"aguas"      AGUAS
"clavar"     CLAVAR
"shute"      SHUTE
"brega"     BREGA
"cabal"      CABAL
"sino"       SINO

/* Operadores */
"="          ASSIGN
"+"          PLUS
"_"          MINUS
"*"          TIMES
"/"          DIVIDE
"<"          LT
">"          GT
"<="         LE
">="         GE
"=="         EQ
"!="         NE

/* Delimitadores */
"("          LPAREN
```

"")"	RPAREN
"{"	LBRACE
"}"	RBRACE
".,"	SEMI
" , "	COMMA

4. Sintaxis de las 10 capacidades obligatorias

#	Capacidad	Sintaxis en Chapin
1	Declaración de variables	va pisto contador;
2	Asignación de valores	contador = 5; (o va pisto contador = 5;)
3	Expresiones aritméticas	contador = a + b * 2 - c / 4;
4	Precedencia de operaciones	* y / antes que + y -; () fuerza el orden
5	Comparaciones	contador < 10, contador == 0, etc.
6	Entrada de datos	aguas "¿Cuántas veces?"; clavar(contador);
7	Salida de datos	shute(contador); o shute("Texto " + contador);
8	Condicional con alternativa	cabal (cond) { ... } sino { ... }
9	Ciclo while	brega (cond) { ... }
10	Comentarios	// línea y /* bloque */

5. Gramática inicial (BNF)

programa	-> lista_stmt
lista_stmt	-> stmt lista_stmt (vacio)
stmt	-> decl_stmt asig_stmt io_stmt if_stmt while_stmt
tipo	-> PISTO QUETZAL
decl_stmt	-> VA tipo ID SEMI VA tipo ID ASSIGN expr SEMI
asig_stmt	-> ID ASSIGN expr SEMI
io_stmt	-> AGUAS CADENA SEMI CLAVAR LPAREN ID RPAREN SEMI SHUTE LPAREN expr RPAREN SEMI
if_stmt	-> CABAL LPAREN cond RPAREN bloque CABAL LPAREN cond RPAREN bloque SINO bloque
while_stmt	-> BREGA LPAREN cond RPAREN bloque
bloque	-> LBRACE lista_stmt RBRACE
cond	-> expr op_rel expr
op_rel	-> LT GT LE GE EQ NE
expr	-> expr PLUS termino expr MINUS termino expr PLUS CADENA termino
termino	-> termino TIMES factor termino DIVIDE factor factor
factor	-> LPAREN expr RPAREN ID ENTERO REAL CADENA MINUS factor

Nota: la regla expr -> expr PLUS CADENA habilita la concatenación simple de texto con valores (usada en shute("Ola: " + contador)). En Bison se resolverá vía chequeo semántico de tipos en la fase de validaciones, no en la gramática pura

6. Precedencia y asociatividad

De menor a mayor precedencia (igual que en Bison, %left de arriba hacia abajo):

Nivel	Operadores	Asociatividad
1 (más baja)	< > <= >= == !=	no asociativos
2	+ - (binarios)	izquierda
3	* /	izquierda
4	- (unario, negación)	derecha
5 (más alta)	()	—

Ejemplo: $2 + 3 * 4$ se evalúa como $2 + (3 * 4) = 14$, y $(2 + 3) * 4 = 20$.

7. Ejemplos válidos (6)

```
// 1. Declaración simple
va pisto contador;

// 2. Declaración con inicialización
va quetzal promedio = 3.5;

// 3. Asignación con expresión aritmética y precedencia
contador = 2 + 3 * 4;

// 4. Entrada de datos con prompt
aguas "¿Cuántas veces quieres bregar?";
clavar(contador);

// 5. Ciclo while con condición y salida
brega (contador < 10) {
    shute("Vuelta: " + contador);
    contador = contador + 1;
}

// 6. Condicional con alternativa
cabal (contador >= 5) {
    shute("¡Cabal, llegamos alto!");
} sino {
    shute("Quedó bajo, va pues.");
}
```

8. Ejemplos inválidos (6)

```
// 1. Declaración sin identificador
va pisto = 5;
// Error: falta el nombre de la variable después del tipo

// 2. Falta punto y coma
contador = 5
shute(contador);
// Error: toda instrucción debe terminar en ';'

// 3. Condición de brega sin paréntesis
brega contador < 10 {
    shute(contador);
}
// Error: la condición debe ir entre '(' y ')'

// 4. Identificador inválido (empieza con dígito)
va pisto 2contador = 0;
// Error: un identificador no puede iniciar con un dígito

// 5. shute sin paréntesis
shute "Hola";
// Error: shute requiere paréntesis: shute("Hola");

// 6. Bloque sin llave de cierre
cabal (contador > 5) {
    shute("Alto");
// Error: falta '}' para cerrar el bloque
```

9. Programa demostrativo

Combina declaración, entrada, aritmética con precedencia, comparación, ciclo, condicional y comentarios.

```

/* Programa demostrativo: Chapin
   Cuenta vueltas de brega y clasifica el resultado */

va pisto contador = 0;
va pisto limite;
va quetzal promedio = 0.0;

aguas "¿Hasta qué número quieres bregar?";
clavar(limite);

brega (contador < limite) {
    shute("Vuelta: " + contador);
    promedio = promedio + contador;
    contador = contador + 1;
}

promedio = promedio / limite;

cabal (promedio > 5) {
    shute("Promedio alto, quedó cabal: " + promedio);
} sino {
    shute("Promedio bajo: " + promedio);
}

// Fin del programa

```

10. Regla de diseño aplicada

Siguiendo la guía del proyecto, la arquitectura de Chapin separará claramente el análisis del lenguaje fuente de la generación de la salida:

- Análisis del lenguaje fuente: Flex (tokens) -> Bison (gramática/AST) -> Validaciones.
- Generación de la salida: IR -> Generador de código C (versión 11), módulo independiente del front-end, para permitir cambiar el lenguaje destino sin tocar el analizador.